

Rapport

Avionique -A-

Team : Amine

Ammar

Introduction :

Ce projet est un contrôleur de vol embarqué (flight computer) destiné à équiper une fusée expérimentale ou un modèle de fusée. Le système combine une carte électronique sur mesure (conçue sous KiCad, importée initialement depuis EasyEDA) et un firmware développé sur microcontrôleur STM32F411 .

L'objectif fonctionnel du système est de mesurer en temps réel l'orientation (attitude : roulis, tangage, lacet) et les paramètres atmosphériques (température, pression, altitude) de l'engin, tout en assurant la journalisation de ces données sur une carte micro-SD et leur diffusion en direct via une liaison USB (émulation de port série / CDC) .

Objectifs identifiés :

- Acquérir en continu l'attitude de la fusée (roulis/tangage/lacet) via une centrale inertielle (IMU).
- Mesurer la température, la pression atmosphérique et estimer l'altitude via un baromètre.
- Enregistrer les données de vol sur une carte SD pour analyse post-vol.
- Transmettre les données en temps réel à un ordinateur de contrôle au sol via USB (utile en phase de test au sol).
- Fournir une plateforme matérielle compacte et légère (communication SPI) adaptée à l'embarquement.

Bibliothèques des composants utilise :

I. STM32F411 :

Le microcontrôleur est piloté via la bibliothèque officielle STM32Cube HAL (Hardware Abstraction Layer) fournie par ST, complétée par la couche CMSIS (définitions bas niveau du cœur Cortex-M4). Cette bibliothèque fournit des fonctions génériques pour piloter chaque périphérique interne sans manipuler directement les registres.

Module HAL utilisé	Fonction dans le projet
RCC (Reset and Clock Control)	Configuration de la PLL et de l'horloge système à 96 MHz à partir du quartz externe
GPIO	Configuration des broches (sorties CS, LED ; broches SPI ; broches USB ; broches oscillateur)
SPI	Pilotage du bus SPI2 en mode maître pour dialoguer avec l'IMU, le baromètre et la carte SD
NVIC / SysTick	Gestion des interruptions processeur et de la base de temps utilisée par HAL_Delay()
USB Device Core	Pile logicielle de bas niveau gérant l'énumération et les échanges USB

MICROCONTRÔLEUR



Rôle : unité de calcul centrale du contrôleur de vol. Elle exécute le firmware, pilote le bus SPI2 partagé (IMU, baromètre, carte SD distingués par leur ligne CS), gère la pile USB (téléométrie) et cadence l'ensemble via une PLL sur quartz externe 25 MHz portée à 96 MHz.

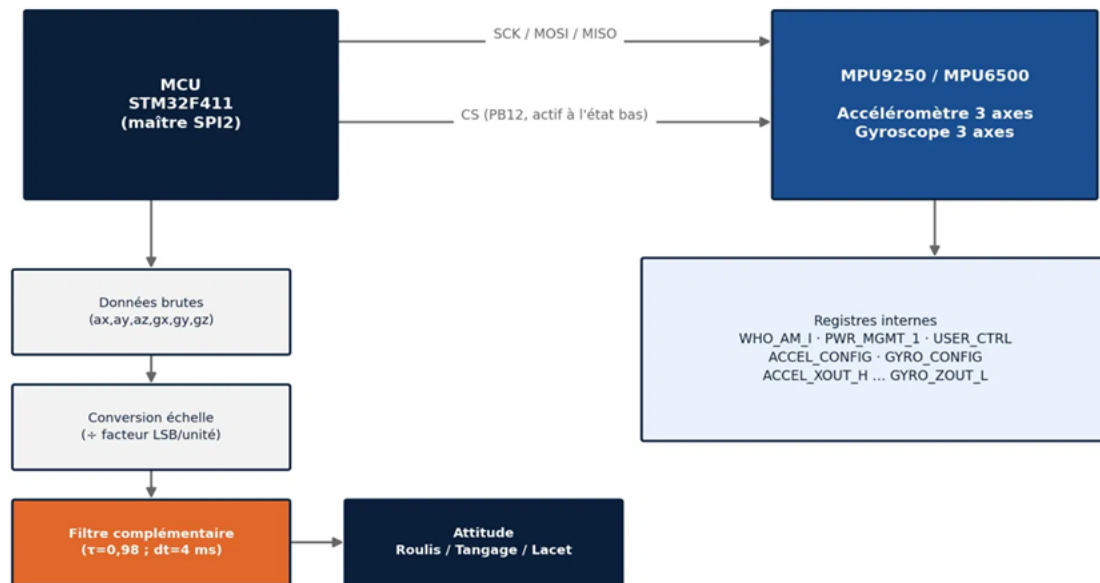
II. MPU9250 :

Le MPU9250/MPU6500 est une centrale inertielle intégrant un accéléromètre 3 axes et un gyroscope 3 axes sur la même puce. L'accéléromètre mesure l'accélération subie sur chaque axe (en g), tandis que le gyroscope mesure la vitesse angulaire de rotation (en degrés par seconde). Le composant expose ses mesures et sa configuration via un ensemble de registres internes accessibles par bus SPI : registre d'identification (WHO_AM_I), registres de configuration des plages de mesure, et registres de données brutes (6 valeurs 16 bits, une par axe et par capteur).

Le projet utilise un pilote dédié, `MPU9250.c / MPU9250.h`, écrit spécifiquement pour ce composant (et non une bibliothèque tierce). Il encapsule toute la communication SPI et le traitement des données dans une structure `MPU9250_t`.

Fonction de la bibliothèques	Rôle
MPU_begin()	Détecte le capteur (lecture WHO_AM_I), le réinitialise, désactive l'I2C au profit du SPI seul, puis applique les plages de mesure choisies
MPU_REG_READ() / MPU_REG_WRITE()	Fonctions bas niveau de lecture/écriture d'un registre via SPI
MPU_writeAccFullScaleRange() / MPU_writeGyroFullScaleRange()	Configurent la sensibilité de l'accéléromètre (± 4 g) et du gyroscope (± 500 °/s) et leurs facteurs de conversion
MPU_readRawData()	Lit en une seule transaction les 6 valeurs brutes (accéléromètre + gyroscope)
MPU_calibrateGyro()	Calcule le biais du gyroscope au repos, par moyenne sur un grand nombre d'échantillons
MPU_readProcessedData()	Convertit les données brutes en unités physiques (g et °/s), en appliquant la correction de biais
MPU_calcAttitude()	Combine accéléromètre et gyroscope (filtre complémentaire) pour calculer l'orientation (roulis, tangage, lacet)

CENTRALE INERTIELLE (IMU)



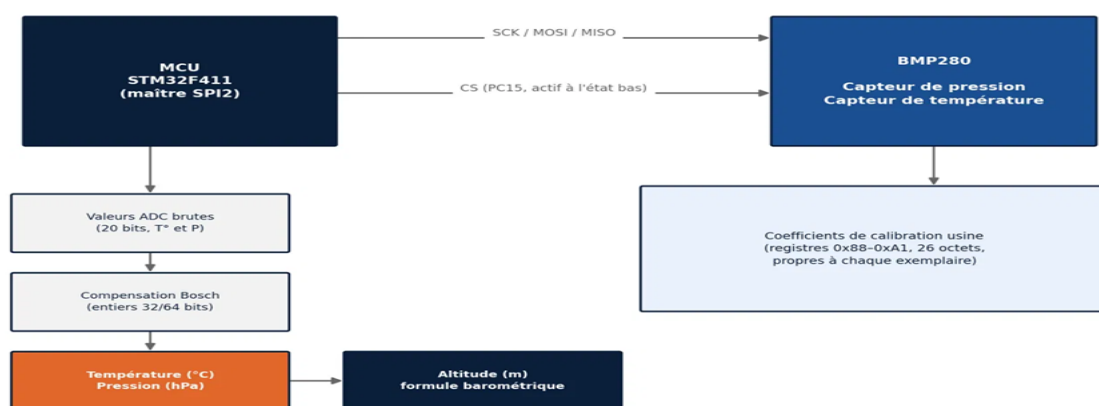
III. **BMP280 :**

Le BMP280 (Bosch Sensortec) est un capteur combiné de pression atmosphérique et de température. Chaque exemplaire du capteur est calibré en usine : des coefficients de correction propres à la puce sont enregistrés dans sa mémoire interne et doivent être lus puis appliqués (compensation) pour transformer les valeurs brutes de sortie du capteur en valeurs physiques exactes. La pression mesurée permet ensuite, via la formule barométrique standard, de calculer une altitude relative à une pression de référence (niveau de la mer).

Le projet utilise un pilote dédié, `BMP280.c / BMP280.h`, qui implémente la communication SPI ainsi que les formules officielles de compensation fournies par Bosch, en arithmétique entière pour limiter la charge de calcul sur le microcontrôleur.

Fonction de la bibliothèque	Rôle
BMP280_Attach()	Associe la structure logicielle du capteur au bus SPI et à sa broche CS
BMP280_ReadChipId() / BMP280_SoftReset()	Vérifient la présence du capteur et le réinitialisent
bmp280_read_calibration()	Lit les 26 octets de coefficients de calibration usine, propres à chaque exemplaire
BMP280_Init()	Séquence complète d'initialisation : vérification, reset, lecture calibration, configuration des sur-échantillonnages et du mode de mesure
BMP280_ReadRaw()	Lit les valeurs brutes (non compensées) de température et de pression
bmp280_compensate_T_int32() / bmp280_compensate_P_int64()	Appliquent les formules de compensation Bosch pour obtenir température et pression exactes
BMP280_ReadFloat()	Renvoie directement température (°C) et pression (hPa) compensées, en virgule flottante
BMP280_CalculateAltitude() / BMP280_ReadAltitude()	Calculent l'altitude à partir de la pression mesurée et d'une pression de référence au niveau de la mer

BAROMÈTRE / ALTIMÈTRE



IV. SdCard :

Une carte micro-SD standard peut fonctionner selon deux protocoles : son protocole natif (bus 1 ou 4 bits) ou un mode compatible SPI, plus simple à mettre en œuvre sur un microcontrôleur ne disposant pas de contrôleur SD dédié c'est ce second mode qui est utilisé ici. En mode SPI, la carte se comporte comme un périphérique esclave classique : elle reçoit des commandes normalisées (CMDx, ACMDx) et répond par des jetons et des blocs de données de 512 octets, l'unité élémentaire de lecture/écriture.

Le projet utilise un pilote dédié, `SdCard.c / SdCard.h`, qui implémente directement le protocole SD en mode SPI (sans passer par une bibliothèque de système de fichiers tierce comme FatFs) : gestion des commandes, des réponses et des jetons de données bas niveau.

Fonction de la bibliothèque	Rôle
SDCARD_Select() / SDCARD_Unselect()	Pilotent la broche CS de la carte SD
SDCARD_Init()	Séquence d'initialisation du protocole SD-SPI : CMD0 (reset), CMD8 (vérification de version/tension), boucle ACMD41 (initialisation), CMD58 (détection carte haute capacité)
SDCARD_GetBlocksNumber()	Lit le registre CSD de la carte pour en déduire sa capacité totale, en blocs de 512 octets
SDCARD_ReadSingleBlock() / SDCARD_WriteSingleBlock()	Lisent ou écrivent un seul bloc de 512 octets (commandes CMD17/CMD24)
SDCARD_ReadBegin/Data/End()	Lecture de plusieurs blocs consécutifs en continu (CMD18), plus efficace pour de grands volumes de données
SDCARD_WriteBegin/Data/End()	Écriture de plusieurs blocs consécutifs en continu (CMD25) mécanisme adapté à un enregistrement de vol prolongé

STOCKAGE -- CARTE MICRO-SD

